

Стартери, логування та документування API (OpenAPI)

Тема 5

Зміст лекції

- Архітектура стартерів та механізм автоконфігурації Spring Boot
- Умовні анотації Conditional та налаштування бінів
- Типізована конфігурація через ConfigurationProperties та профілі
- Основи та архітектура підсистеми логування (SLF4J, Logback)
- Рівні логування, апендери, шаблони та структурний формат JSON
- Контекст діагностики MDC та маскування чутливих даних
- Специфікація OpenAPI та генерація документації з SpringDoc
- Анотації Swagger для моделювання контрактів та типізації
- Автоматизація тестування API через Postman та Newman

Управління залежностями та концепція BOM

У складних enterprise-проєктах підключення сторонніх бібліотек вручну спричиняє конфлікти транзитивних версій (Dependency Hell).

- Інструмент Bill of Materials (BOM) централізовано фіксує узгоджені версії сотень сумісних бібліотек у секції `dependencyManagement`.
- Базовий BOM у Spring Boot позбавляє потреби вручну прописувати тег `<version>` для підтримуваних артефактів.
- Оновлення однієї версії батьківського BOM автоматично підтягує сумісні релізи драйверів баз даних, серіалізаторів та веб-серверів.

Документація: [Spring Boot Dependency Management & BOM](#)

Приклад підключення BOM у системі Maven

```
<dependencyManagement>
  <dependencies>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-dependencies</artifactId>
      <version>4.1.0</version>
      <type>pom</type>
      <scope>import</scope>
    </dependency>
  </dependencies>
</dependencyManagement>

<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>
</dependencies>
```

Пояснення налаштування BOM у Maven

- Секція `dependencyManagement` імпортує узгоджений перелік версій без їх включення в кінцевий артефакт.
- Параметр `scope=import` з типом `pom` дозволяє підключати кілька незалежних BOM-файлів без спадкування батьківського POM.
- У блоці `dependencies` версії бібліотек не вказуються та автоматично синхронізуються з релізом Spring Boot BOM.
- Зміна версії в одній точці автоматично оновлює всі транзитивні бібліотеки в проєкті.

Приклад підключення BOM у системі Gradle

```
dependencies {  
    // Імпорт специфікації версій BOM через нативну платформу Gradle  
    implementation platform('org.springframework.boot:spring-boot-dependencies:4.1.0')  
  
    // Оголошення залежностей без зазначення версій  
    implementation 'org.springframework.boot:spring-boot-starter-web'  
    implementation 'org.springframework.boot:spring-boot-starter-validation'  
    testImplementation 'org.springframework.boot:spring-boot-starter-test'  
}
```

Пояснення роботи платформи BOM у Gradle

- Метод `platform()` імпортує правила та обмеження версій із файлу BOM на рівні нативного механізму Gradle.
- Усі транзитивні бібліотеки автоматично узгоджуються за версіями без потреби сторонніх плагінів управління залежностями.
- Метод `enforcedPlatform()` дозволяє примусово зафіксувати версії у випадку конфліктів зі сторонніми транзитивними залежностями.
- Синтаксис забезпечує однакову поведінку управління версіями як у Maven, так і в Gradle.

Концепція Spring Boot Starters

Стартер є спеціальним дескриптором залежностей, що містить повний набір бібліотек для вирішення конкретної інженерної задачі:

- Замість підключення десятка окремих jar-файлів (Tomcat, Spring MVC, Jackson, Validation) розробник додає єдиний артефакт `spring-boot-starter-web`.
- Офіційні стартери екосистеми Spring Boot завжди іменуються за шаблоном `spring-boot-starter-*`.
- Сторонні та власні бібліотеки мають використовувати конвенцію `*-spring-boot-starter` для запобігання конфліктам просторів імен.

Архітектурний шаблон стартера з двох модулів

Офіційний стандарт Spring Boot рекомендує розділяти розробку стартера на два окремі модулі:

- **xxx-spring-boot-autoconfigure** : містить класи конфігурацій `@AutoConfiguration` , біни сервісів, класи параметрів `@ConfigurationProperties` та файл `AutoConfiguration.imports` .
- **xxx-spring-boot-starter** : порожній модуль-агрегатор, що містить виключно файл збірки (`pom.xml` або `build.gradle`) із переліком необхідних транзитивних залежностей та модулем `autoconfigure`.
- Запобігає випадковому забрудненню `classpath` клієнтських застосунків зайвими залежностями та розділяє логіку автоконфігурації й постачання бібліотек.

Механізм автоконфігурації в Spring Boot 4.1

Автоконфігурація аналізує вміст classpath, активні налаштування та наявні біни під час старту застосунку:

- Робота підсистеми ініціюється мета-анотацією `@EnableAutoConfiguration`, яка входить до складу `@SpringBootApplication`.
- Спеціальний селектор імпорту зчитує перелік зареєстрованих конфігураційних класів та передає їх на аналіз контексту.
- Створення інфраструктурних компонентів відбувається за принципом Convention over Configuration.

Декларація файлу `AutoConfiguration.imports`

Шлях до конфігураційного дескриптора стартера:

```
META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports
```

```
com.example.camera.config.CameraAutoConfiguration  
com.example.camera.config.CameraSecurityAutoConfiguration  
com.example.camera.config.CameraStorageAutoConfiguration
```

Пояснення механізму імпорту автоконфігурацій

- Підхід усуває повільне сканування пакетів рефлексією на етапі старту віртуальної машини.
- Повністю замінено застарілий файл `spring.factories`, починаючи зі Spring Boot 3 та у 4.1.
- Кожен рядок містить повне кваліфіковане ім'я окремого конфігураційного класу, що завантажується детерміновано.
- Спрощує сумісність із компіляцією в нативний образ GraalVM AOT.

Документація: [Spring Boot Developing Auto-configuration](#)

Приклад структури класу автоконфігурації

```
@AutoConfiguration
public class CameraAutoConfiguration {

    @Bean
    @ConditionalOnMissingBean
    public CameraStreamService cameraStreamService(CameraProperties properties) {
        return new DefaultCameraStreamService(properties.getStreamEndpoint());
    }
}
```

Пояснення структури класу автоконфігурації

- Анотація `@AutoConfiguration` позначає конфігураційний клас вищого рівня для механізму імпорту.
- Параметр `proxyBeanMethods = false` увімкнено за замовчуванням для усунення накладних витрат CGLIB-проксі.
- Оголошені біни реєструються в контексті лише за умови виконання накладених умовних предикатів.
- Методи повертають інфраструктурні компоненти за принципом Convention over Configuration.

Приклад налаштування порядку автоконфігурацій

```
@AutoConfiguration(  
    after = DataSourceAutoConfiguration.class,  
    before = TransactionAutoConfiguration.class  
)  
public class CameraAuditAutoConfiguration {  
  
    @Bean  
    @ConditionalOnBean(DataSource.class)  
    public CameraAuditRepository cameraAuditRepository(DataSource dataSource) {  
        return new JdbcCameraAuditRepository(dataSource);  
    }  
}
```

Пояснення черговості виконання автоконфігурацій

- Автоконфігурації виконуються не хаотично, а згідно з детермінованим графом залежностей компонентів.
- Якщо стартер залежить від уже створених інфраструктурних бінів (DataSource, Security), необхідно явно задавати черговість.
- Анотація `@AutoConfiguration` підтримує атрибути `after`, `afterName`, `before` та `beforeName` для гнучкого позиціонування в ланцюжку ініціалізації.
- Запобігає падінню додатку з помилкою `NoSuchBeanDefinitionException` під час старту.

Приклад умовної реєстрації через ConditionalOnClass

```
@AutoConfiguration
@ConditionalOnClass(name = "com.fasterxml.jackson.databind.ObjectMapper")
public class JsonPayloadAutoConfiguration {

    @Bean
    @ConditionalOnMissingBean
    public PayloadSerializer jsonPayloadSerializer(ObjectMapper objectMapper) {
        return new JacksonPayloadSerializer(objectMapper);
    }
}
```

Пояснення роботи анотації ConditionalOnClass

- Анотація `@ConditionalOnClass` перевіряє фізичну присутність зазначеного класу в завантажувачі класів.
- Якщо відповідна бібліотека відсутня в залежностях проєкту, вся конфігурація безпечно ігнорується без помилок збірки.
- Дозволяє розробляти універсальні стартери з підтримкою альтернативних бібліотек (Jackson, Gson, Protobuf).
- Усуває жорстку прив'язку модуля до єдиного постачальника сторонньої реалізації.

Приклад умовної активації через ConditionalOnProperty

```
@Bean
@ConditionalOnProperty(
    prefix = "camera.stream",
    name = "enabled",
    havingValue = "true",
    matchIfMissing = false
)
public StreamRecorder streamRecorder() {
    return new FfmpegStreamRecorder();
}
```

Пояснення параметрів ConditionalOnProperty

- Анотація `@ConditionalOnProperty` перевіряє значення параметра у файлах `application.yml` або змінних оточення.
- Параметр `havingValue` задає очікуване строкове значення для успішної активації біна.
- Параметр `matchIfMissing` визначає поведінку у випадку повної відсутності ключа в зовнішній конфігурації.
- Дозволяє розробникам вмикати або вимикати важкі підсистеми простим прапорцем у конфігурації.

Приклад оголошення заміщуваного біна за замовчуванням

```
@Bean
@ConditionalOnMissingBean(StreamEncoder.class)
public StreamEncoder defaultH264Encoder() {
    return new HardwareAcceleratedH264Encoder();
}
```

Пояснення механізму заміщення бінів

- Анотація `@ConditionalOnMissingBean` є ключовим інструментом побудови гнучких бібліотек.
- Дозволяє розробнику прикладного проєкту оголосити власну реалізацію сервісу з анотацією `@Bean`.
- Якщо користувацький бін уже присутній у контексті, фабричний метод автоконфігурації не виконується.
- Забезпечує постачання надійної поведінки за замовчуванням із можливістю повного кастомізування без форку бібліотеки.

Команда запуску діагностики автоконфігурацій

```
# Активація режиму детальної діагностики стартерів у консолі  
java -jar cctv-app.jar --debug
```

Пояснення структури звіту умов

- Запуск застосунку з прапорцем `--debug` або налаштуванням `debug=true` в `application.yml` активує звіт `CONDITIONS EVALUATION REPORT`.
- **Positive matches:** перелік успішно виконаних умов, через які класи та біни додано до контексту Spring.
- **Negative matches:** причини відхилення конфігурацій (відсутність класу в classpath, невідповідність `@ConditionalOnProperty`).
- **Exclusions та Unconditional classes:** явно виключені конфігурації та біни без накладених умов.

Повний цикл створення власного стартера

Проектування власного бібліотечного модуля складається з чітких кроків:

- Визначення доменної логіки та сервісів, які автоматично надаються користувачам стартера.
- Створення класу властивостей `@ConfigurationProperties` для параметризації поведінки.
- Реалізація конфігураційного класу з анотаціями умовного створення бінів.
- Реєстрація класу в файлі `META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports`.
- Публікація стартера як спільного модуля для команд розробки.

Концепція зовнішньої конфігурації застосунку

Відповідно до методології Twelve-Factor App конфігурація повинна бути суворо відокремлена від коду:

- Один і той самий скомпільований jar-артефакт розгортається на локальному комп'ютері, в CI-середовищі та на продакшні без перекомпіляції.
- Spring Boot дозволяє читати параметри з різноманітних джерел (змінні ОС, файли YAML, системні властивості).
- Значення автоматично зв'язуються з типізованими класами в коді за допомогою механізму `@ConfigurationProperties`.
- Будь-який параметр може бути перевизначений під час запуску в контейнерах Docker або Kubernetes.

Приклад типізованих налаштувань на базі Java Record

```
@ConfigurationProperties(prefix = "storage.s3")
@Validated
public record S3StorageProperties(
    @NotBlank String bucket,
    @NotBlank String region,
    @Min(1) @Max(100) int maxConnections,
    Duration timeout
) {}
```

Пояснення переваг незмінюваних налаштувань у Records

- Використання Java Records забезпечує незмінність (immutability) конфігураційного стану після ініціалізації.
- Анотація `@ConfigurationProperties` зв'язує префікс YAML або properties-файлу з компонентами запису.
- Канонічний конструктор Record автоматично використовується для безпечного створення об'єкта.
- Підтримує автоматичне перетворення часових інтервалів (наприклад, `timeout: 5s` у тип `Duration`).

Приклад структури конфігураційного файлу YAML

```
storage:  
  s3:  
    bucket: "camera-recordings-archive"  
    region: "eu-central-1"  
    max-connections: 20  
    timeout: 5s
```

Пояснення валідації властивостей за принципом Fail-Fast

- Анотація `@Validated` запускає валідацію Jakarta Validation під час створення біна.
- Помилка у форматі або відсутність обов'язкового параметра миттєво зупиняє запуск додатку (Fail-Fast).
- У консоль виводиться зрозуміле діагностичне повідомлення замість пізніх `NullPointerException` під навантаженням.
- Підтримуються перевірки `@NotBlank`, `@Min`, `@Max`, `@Pattern`, `@Positive` та кастомні валідатори.

Головні джерела зовнішньої конфігурації (Рантайм)

Spring Boot визначає суворий порядок пріоритетів для перевизначення властивостей:

- 1. Аргументи командного рядка: `--server.port=9090` (мають найвищий пріоритет).
- 2. Системні властивості JVM: `-Dserver.port=9090`.
- 3. Змінні оточення ОС: `SERVER_PORT=9090` або `SPRING_APPLICATION_JSON`.

Дозволяє динамічно перевизначати параметри в Docker та Kubernetes без перекомпіляції.

Головні джерела зовнішньої конфігурації (Файли)

Порядок пріоритетів для конфігураційних файлів та значень за замовчуванням:

- 4. Профільно-специфічні файли: `application-{profile}` (`.properties` або `.yml`).
- 5. Базовий файл налаштувань: `application` (`.properties` або `.yml`) у ресурсах проєкту.
- 6. Значення за замовчуванням у коді: дефолти у полях Java Records чи `@Value` .

Це лише основні джерела: повна специфікація Spring Boot налічує 17 рівнів пріоритету.

Офіційна документація: [Spring Boot External Config](#)

Робота з профілями середовищ Spring Profiles

Профілі дозволяють розділяти конфігурації для локальної розробки, тестового середовища та промислової експлуатації:

- Активація потрібного профілю здійснюється параметром `--spring.profiles.active=dev` або змінною `SPRING_PROFILES_ACTIVE`.
- Застосунок завантажує спільний файл `application`, поверх якого накладаються значення з `application-dev` (`.properties` або `.yml`).
- Біни та конфігураційні класи можуть активуватися вибірково за допомогою анотації `@Profile("prod")`.

Приклад тестування автоконфігурації через ApplicationContextRunner

```
class CameraAutoConfigurationTest {  
  
    private final ApplicationContextRunner runner = new ApplicationContextRunner()  
        .withConfiguration(AutoConfigurations.of(CameraAutoConfiguration.class));  
  
    @Test  
    void shouldRegisterBeanWhenPropertyEnabled() {  
        this.runner  
            .withPropertyValues("camera.stream.enabled=true")  
            .run(context -> {  
                assertThat(context).hasSingleBean(StreamRecorder.class);  
            });  
    }  
}
```

Пояснення тестування за допомогою ApplicationContextRunner

- Утиліта `ApplicationContextRunner` ініціалізує контекст за мілісекунди без запуску веб-сервера Tomcat.
- Дозволяє перевіряти умови додавання або пропуску бінів за різних комбінацій значень конфігурації.
- Забезпечує високу швидкість виконання модульних тестів стартера у пайплайнах неперервної інтеграції.
- Надає багатий набір асертів `assertThat(context)` для перевірки наявності або відсутності бінів.

[Документація: Testing Auto-configurations](#)

Висновки розділу про стартери та автоконфігурацію

- Стартери та BOM усувають конфлікти версій та стандартизують стек залежностей проєкту.
- Механізм автоконфігурації спирається на детермінований файл `AutoConfiguration.imports` та умовні анотації.
- Налаштування на базі незмінюваних Java Records та валідації забезпечують надійний старт системи.
- Швидкі тести на базі `ApplicationContextRunner` гарантують коректність роботи бібліотечних компонентів.

Підсистема логування у Spring Boot та Java 25

Організація спостережуваності та збору подій у сучасних бекенд-системах:

- Логування є фундаментом діагностики та спостережуваності (Observability).
- Сучасний стек спирається на фасад SLF4J та гнучкий рушій Logback.
- Використання структурованих форматів спрощує централізований збір логів.

Архітектура та призначення фасаду SLF4J

Simple Logging Facade for Java (SLF4J) є інтерфейсною абстракцією над рушіями логування:

- Бібліотеки та сервіси викликають методи інтерфейсу `org.slf4j.Logger`, не прив'язуючись до конкретної реалізації.
- Рушій логування (Logback, Log4j2) підключається на етапі збірки та може замінюватися без редагування бізнес-коду.
- Запобігає конфліктам, коли різні сторонні залежності намагаються використовувати несумісні логери.

Рушій логування Logback за замовчуванням

Фреймворк Spring Boot постачається з готовою конфігурацією Logback:

- Logback є прямою сучасною реалізацією інтерфейсів SLF4J без проміжних адаптерів.
- Забезпечує високу продуктивність запису та низьке споживання оперативної пам'яті.
- Підтримує динамічне перезавантаження конфігурації без перезапуску застосунку та фільтрацію повідомлень.

Документація: [Spring Boot Logging](#)

Архітектурні відмінності Logback та Log4j2

Вибір рушія логування впливає на інтеграцію з екосистемою та структуру бібліотек:

- **Logback:** нативна реалізація SLF4J від автора концепції, не потребує додаткових перехідників у Spring Boot.
- **Log4j2:** має власний розвинений API (`log4j-api`) та використовує міст-адаптер (`log4j-slf4j2-impl`) для сумісності з SLF4J.
- **Модульність:** Log4j2 використовує власну плагінну систему на базі анотацій з індексацією під час компіляції.
- **Підтримка у фреймворку:** обидва рушії мають офіційні стартери Spring Boot та однакову підтримку з боку Actuator.

Продуктивність та асинхронне логування в Log4j2

Ключова перевага Log4j2 розкривається у високонавантажених багатопотокових системах:

- **Асинхронний рушій:** Logback використовує `ArrayBlockingQueue`, тоді як Log4j2 базується на lock-free кільцевому буфері **LMAX Disruptor**.
- **Пропускна здатність:** швидкість запису асинхронних логів у Log4j2 у рази вища при одночасній роботі сотень потоків.
- **Затримка (Latency):** забезпечує стабільно низький час відгуку без блокувань робочих потоків веб-сервера.
- **Garbage-free підхід:** Log4j2 повторно використовує буфери повідомлень, мінімізуючи навантаження на Garbage Collector.

Конфігураційні можливості та плагіни

Гнучкість адміністрування та доступні формати декларації:

- **Формати файлів:** Logback обмежується XML (`logback-spring.xml`) та Groovy; Log4j2 підтримує XML, JSON, YAML та Properties.
- **Фільтрація подій:** Log4j2 надає ширший каталог фільтрів (ContextMapFilter, RegexFilter, BurstFilter для захисту від флуду).
- **Прямі апендери:** Log4j2 містить готові оптимізовані апендери до Apache Kafka, Flume, NoSQL без сторонніх обгорт.
- **Гаряче перезавантаження:** обидва рушії підтримують перерачитування конфігурації без перезапуску застосунку.

Приклад переходу на Log4j2 у збірнику Maven

```
<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
    <exclusions>
      <exclusion>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-logging</artifactId>
      </exclusion>
    </exclusions>
  </dependency>

  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-log4j2</artifactId>
  </dependency>
</dependencies>
```

Пояснення налаштування залежностей у Maven

- Необхідно явно виключити `spring-boot-starter-logging` із веб-стартера для запобігання конфлікту провайдерів.
- Стартер `spring-boot-starter-log4j2` містить оптимізовані адаптери SLF4J та ядро `log4j-core`.
- Прикладний код не змінюється: класи продовжують викликати анотацію `@Slf4j` або інтерфейс `org.slf4j.Logger`.
- Конфігурація для середовищ розміщується у файлі `log4j2-spring.xml` у каталозі ресурсів.

Документація: [Spring Boot Configure Log4j2](#)

Приклад переходу на Log4j2 у збірнику Gradle

```
configurations {  
    all {  
        exclude group: 'org.springframework.boot', module: 'spring-boot-starter-logging'  
    }  
}  
  
dependencies {  
    implementation 'org.springframework.boot:spring-boot-starter-web'  
    implementation 'org.springframework.boot:spring-boot-starter-log4j2'  
}
```

Пояснення конфігурації виключень у Gradle

- Блок `configurations.all` централізовано виключає Logback з усіх підключених модулів та залежностей.
- Усуває ризик появи подвійного зв'язування реалізацій SLF4J через транзитивні сторонні бібліотеки.
- Декларація `spring-boot-starter-log4j2` активує відповідну підсистему автоконфігурації Spring Boot.
- Фреймворк автоматично застосовує профільні налаштування логування для активного профілю оточення.

Практичний вибір між Logback та Log4j2

Архітектурні критерії вибору рушія для проєкту:

- **Logback (вибір за замовчуванням):** стандартні REST-сервіси, мікросервіси та CRUD-додатки. Забезпечує нульову конфігурацію, нативну сумісність зі SLF4J та достатню продуктивність для 95% enterprise-задач.
- **Log4j2 (вибір для HighLoad):** високонавантажені системи, фінтех та обробка подій у реальному часі з вимогами до стабільно низької затримки.
- **Асинхронна перевага:** Log4j2 необхідний там, де потрібна екстремальна пропускна здатність на базі lock-free буфера LMAX Disruptor без навантаження на збирач сміття.
- **Екосистема та формати:** Log4j2 обирають за потреби конфігурації у форматі YAML або JSON, а також прямого виводу в Apache Kafka.

Архітектурна взаємодія компонентів логування

Процес обробки журналу подій складається з трьох ключових рівнів:

1. **Logger**: приймає повідомлення від прикладного коду разом із рівнем важливості та аргументами.
2. **Appender**: відповідає за фізичне місце призначення логів (консоль, локальний файл, мережевий сокет, Kafka).
3. **Encoder / Layout**: перетворює об'єкт події на байтовий масив або текстовий рядок визначеного формату (текст, JSON).

Приклад оголошення та виклику логера SLF4J

```
public class CameraManagementService {  
    private static final Logger log = LoggerFactory.getLogger(CameraManagementService.class);  
  
    public void registerCamera(CameraRegistrationCommand command) {  
        log.info("Реєстрація нової камери з назвою: {}", command.name());  
        // виконання бізнес-логіки  
    }  
}
```

Пояснення роботи логера через фасад SLF4J

- Статичне поле `log` гарантує створення єдиного екземпляра логера на весь завантажений клас.
- Замість ручного оголошення дозволяється використовувати анотацію `@Slf4j` бібліотеки Lombok.
- Використання фасаду захищає код сервісу від прямої прив'язки до реалізації Logback чи Log4j2.
- Виклик методів логування здійснюється через параметризовані шаблони без конкатенації рядків.

Рівні логування та матриця їх застосування

Правильний вибір рівня критичності запобігає засміченню журналу:

- **ERROR:** невідновна помилка в роботі системи, що вимагає негайного втручання команди підтримки (відмова БД, втрата з'єднання).
- **WARN:** нештатна ситуація, яку вдалося обробити через резервний сценарій (повторний запит, тимчасовий тайм-аут).
- **INFO:** значуща бізнес-подія життєвого циклу (запуск сервісу, оформлення замовлення, реєстрація користувача).
- **DEBUG:** деталі внутрішньої оркестрації для розробників під час локального тестування (вхідні параметри, проміжні обчислення).
- **TRACE:** вичерпний покроковий вивід низькорівневих пакетів та циклів.

Приклад конфігурації рівнів логування в YAML

```
logging:  
  level:  
    root: INFO  
    org.springframework.web: WARN  
    org.hibernate.SQL: DEBUG  
    org.hibernate.orm.jdbc.bind: TRACE  
    com.example.camera: DEBUG
```

Пояснення налаштування ієрархії рівнів логування

- Рівень `root` задає політику за замовчуванням для всіх компонентів та бібліотек застосунку.
- Для власних пакетів бізнес-логіки встановлюється детальний рівень без засмічення журналу логами фреймворку.
- Ієрархія пакетів дозволяє вмикати детальне трасування SQL-запитів Hibernate або REST-викликів вибірково.
- Підтримує швидку зміну рівнів для локального налагодження складних дефектів.

Приклад параметризації повідомлень логера

```
// Некоректно: конкатенація рядків виконується ЗАВЖДИ, навіть якщо DEBUG вимкнено
log.debug("Знімок камери " + camera.getId() + " розміром " + image.getSize());

// Коректно: рядок формується тільки тоді, коли рівень DEBUG дійсно активний
log.debug("Знімок камери {} розміром {}", camera.getId(), image.getSize());

// Ледаче обчислення важких операцій через лямбда-вирази
log.debug("Повний дамп кадру: {}", (Supplier<String>) image::generateDiagnostics);
```

Пояснення впливу параметризації на продуктивність

- Пряма конкатенація рядків оператором `+` створює об'єкти в пам'яті навіть тоді, коли рівень DEBUG вимкнено.
- Використання заповнювачів `{}` усуває непотрібне виділення пам'яті в купі JVM під час роботи на рівні INFO.
- Механізм передачі постачальників `Supplier<String>` відкладає обчислення діагностики до фактичного запису.
- Знижує навантаження на Garbage Collector у високонавантажених сервісах обробки запитів.

Приклад налаштування RollingFileAppender у Logback

```
<appender name="FILE" class="ch.qos.logback.core.rolling.RollingFileAppender">
  <file>logs/application.log</file>
  <rollingPolicy class="ch.qos.logback.core.rolling.SizeAndTimeBasedRollingPolicy">
    <fileNamePattern>logs/archived/app-%d{yyyy-MM-dd}.%i.log.gz</fileNamePattern>
    <maxFileSize>50MB</maxFileSize>
    <maxHistory>30</maxHistory>
    <totalSizeCap>5GB</totalSizeCap>
  </rollingPolicy>
  <encoder>
    <pattern>%d{ISO8601} [%thread] %-5level %logger{36} - %msg%n</pattern>
  </encoder>
</appender>
```

Пояснення параметрів політики ротації логів

- Політика `SizeAndTimeBasedRollingPolicy` забезпечує нарізку логів при досягненні ліміту розміру або зміні доби.
- Параметр `maxFileSize` захищає сервер від розростання поодиноких гігабайтних файлів логів.
- Параметри `maxHistory` та `totalSizeCap` автоматично очищають старі архіви при вичерпанні дискової квоти.
- Автоматичне стиснення в архів `.gz` суттєво економить простір на серверах постійного моніторингу.

Приклад структурованого журналу у форматі JSON

```
{  
  "@timestamp": "2026-09-30T10:15:30.123Z",  
  "log.level": "INFO",  
  "service.name": "camera-service",  
  "trace.id": "4bf92f3577b34da6a3ce929d0e0e4736",  
  "message": "Камеру підключено до відеопотоку",  
  "camera.id": 1042  
}
```

Пояснення переваг формату JSON для логів

- Логи у форматі JSON готові до індексації в Elasticsearch, Loki, Datadog або CloudWatch без регулярних виразів.
- Spring Boot 4.1 надає нативну підтримку structured logging через налаштування конфігурації.
- Дозволяє будувати швидкі аналітичні запити за окремими полями моделі без додаткового навантаження на процесор.
- Відокремлює службові метадані від тексту інформаційного повідомлення.

Документація: [Spring Boot Structured Logging](#)

Приклад збереження параметрів у контексті MDC

```
MDC.put("traceId", UUID.randomUUID().toString());  
log.info("Початок обробки транзакції");
```

Пояснення принципу роботи контексту MDC

- Mapped Diagnostic Context (MDC) дозволяє прив'язувати контекстні дані до поточного потоку виконання.
- У багатопотоковому середовищі запити від тисяч клієнтів перемішуються в спільному журналі.
- MDC використовує внутрішній механізм `ThreadLocal` для зберігання асоціативного масиву параметрів.
- Значення з MDC автоматично додаються до кожного рядка логу без ручної передачі аргументів у виклики.

[Документація: SLF4J MDC Manual](#)

Приклад веб-фільтра для генерації та передачі Trace ID

```
@Component
public class TraceIdFilter extends OncePerRequestFilter {

    @Override
    protected void doFilterInternal(HttpServletRequest request,
                                    HttpServletResponse response,
                                    FilterChain filterChain) throws ServletException, IOException {

        String traceId = request.getHeader("X-Trace-Id");
        if (traceId == null || traceId.isBlank()) {
            traceId = UUID.randomUUID().toString();
        }

        MDC.put("traceId", traceId);
        response.setHeader("X-Trace-Id", traceId);
        try {
            filterChain.doFilter(request, response);
        } finally {
            MDC.remove("traceId");
        }
    }
}
```

Пояснення роботи фільтра трасування `OncePerRequestFilter`

- Базовий клас `OncePerRequestFilter` гарантує виконання логіки рівно один раз для кожного вхідного HTTP-запиту.
- Фільтр перевіряє наявність вхідного заголовка `X-Trace-Id` або генерує новий унікальний UUID.
- Ідентифікатор записується в заголовок HTTP-відповіді для наскрізного узгодження з клієнтом.
- Обов'язковий блок `finally` гарантує очищення контексту навіть при аварійному завершенні обробки запиту.

Обов'язкове очищення контексту в пулах потоків

Нехтування очищенням MDC призводить до витоку даних між клієнтами:

- Веб-сервер Tomcat повторно використовує потоки з пулу для обслуговування наступних HTTP-запитів.
- Якщо `traceId` або ідентифікатор користувача не видалено в блоці `finally`, наступний запит випадкового клієнта отримає чужий контекст.
- Правилком для будь-якого розробника є виклик `MDC.remove()` або `MDC.clear()` у секції завершення операції.

Приклад виключення паролів з текстового представлення

```
public record UserCredentials(  
    String username,  
    @ToString.Exclude String password  
) {}
```

Пояснення правил захисту чутливих даних у логах

- Логування конфіденційної інформації суворо заборонено стандартами безпеки (GDPR, PCI-DSS).
- Заборонено виводити відкриті паролі, CVC-коди карток, сесійні токени та конфіденційні персональні дані.
- Необхідно налаштовувати спеціальні маскувальні конвертери або санітизувати DTO перед передачею в логер.
- При виведенні об'єктів методом `toString()` поля з паролями мають обов'язково замінюватися на маску.

Приклад коректного та помилкового логування помилок

```
// Некоректно: втрачено причину збою та повний стек виклику
try {
    paymentClient.charge(order);
} catch (Exception e) {
    log.error("Помилка оплати: " + e.getMessage());
}

// Коректно: об'єкт винятку передається останнім аргументом для друку stack trace
try {
    paymentClient.charge(order);
} catch (Exception e) {
    log.error("Помилка виконання платежу для замовлення {}", order.id(), e);
    throw new PaymentProcessingException(order.id(), e);
}
```

Пояснення культури передачі винятків у логер

- Проста конкатенація рядка `e.getMessage()` безповоротно стирає ланцюжок викликів і першопричину збою.
- Об'єкт винятку обов'язково передається останнім параметром методу логера без заповнювача `{}`.
- Неприпустимо одночасно логувати помилку та прокидати її без змін, оскільки це спричиняє лавиноподібне дублювання логів.
- Кожен шар системи повинен або обробити виняток, або збагатити його контекстом і прокинути далі.

Висновки розділу про журналювання

- Фасад SLF4J забезпечує єдиний інтерфейс для бізнес-коду та захищає від залежності від конкретної бібліотеки.
- Конфігурація рівнів через `application.yml` та параметризація рядків оптимізують продуктивність.
- Контекст MDC у парі з `OncePerRequestFilter` створює фундамент для розподіленого трасування запитів.
- Дотримання вимог безпеки запобігає витоку персональних даних у незахищені файли журналів.

Документування REST API через Swagger та OpenAPI

Контрактно-орієнтована розробка та тестування веб-інтерфейсів:

- Документація є обов'язковим контрактом між бекендом та клієнтськими застосунками.
- Специфікація OpenAPI забезпечує машинне читання та інтерактивне тестування.
- Інструменти Swagger UI та Postman прискорюють розробку та автоматизують верифікацію.

Співвідношення стандартів OpenAPI та Swagger

Важливо чітко розрізняти специфікацію та супутні інструменти:

- **OpenAPI Specification (OAS):** відкритий міжнародний стандарт під егідою Linux Foundation для опису RESTful API у форматах JSON та YAML.
- **Swagger:** набір комерційних та open-source інструментів від компанії SmartBear для роботи з цим стандартом.
- **Swagger UI:** інтерактивна веб-сторінка, що зчитує специфікацію OpenAPI та дозволяє тестувати запити безпосередньо з браузера.
- **Swagger Editor / Codegen:** засоби проєктування схеми та генерації клієнтських бібліотек.

Офіційна специфікація: [OpenAPI Specification v3.1.0](#)

Порівняння стратегій Code-First та Contract-First

В індустрії використовуються дві протилежні стратегії розробки:

- **Contract-First (Specification-First):**

Спочатку архітектори описують YAML-схему OpenAPI. Команди фронтенду та бекенду одночасно починають розробку за затвердженням контрактом.

- **Code-First (Implementation-First):**

Розробники пишуть класи контролерів та DTO у Spring Boot, додають анотації, а інструмент автоматично генерує актуальну специфікацію з вихідного коду.

Для швидкої розробки у Spring Boot стандартом де-факто є підхід Code-First за допомогою SpringDoc.

Приклад налаштування плагіна OpenAPI Generator

```
<plugin>
  <groupId>org.openapitools</groupId>
  <artifactId>openapi-generator-maven-plugin</artifactId>
  <version>7.12.0</version>
  <configuration>
    <inputSpec>${project.basedir}/src/main/resources/api.yaml</inputSpec>
    <generatorName>spring</generatorName>
    <configOptions>
      <interfaceOnly>true</interfaceOnly>
      <useSpringBoot3>true</useSpringBoot3>
    </configOptions>
  </configuration>
</plugin>
```

Пояснення переваг генерації коду з контракту

- У підході Contract-First специфікація `openapi.yaml` виступає єдиним джерелом правди для всіх команд.
- Плагін `openapi-generator-maven-plugin` під час збірки автоматично генерує Java-інтерфейси контролерів та DTO.
- Розробник бекенду реалізує інтерфейс, виключаючи ручне створення моделей та помилки в типах даних.
- Зміни у схемі миттєво викликають помилки компіляції, гарантуючи сувору відповідність коду специфікації.

[Документація: OpenAPI Generator Maven Plugin](#)

Приклад підключення залежності SpringDoc OpenAPI

```
<dependency>  
  <groupId>org.springdoc</groupId>  
  <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>  
  <version>2.8.5</version>  
</dependency>
```

Пояснення інтеграції SpringDoc у Spring Boot

- Бібліотека аналізує мапінги контролерів, типи параметрів методів та моделі відповідей під час запуску додатку.
- За адресою `/v3/api-docs` автоматично генерується повна JSON-специфікація стандарту OpenAPI 3.1.
- За адресою `/swagger-ui/index.html` відкривається інтерактивний веб-інтерфейс для ручного тестування.
- Не вимагає додаткових сторонніх серверів чи ручного оновлення документації.

Офіційна документація: [SpringDoc OpenAPI](#)

Структура документа специфікації OpenAPI

Згенерований документ складається з обов'язкових секцій:

- `openapi` : версія використовуваного стандарту (наприклад, 3.1.0).
- `info` : метадані сервісу (назва, версія релізу, опис, контакти автора).
- `servers` : перелік базових адрес (локальне середовище, стейджинг, прод).
- `paths` : каталог усіх маршрутів із методами (GET, POST), параметрами та відповідями.
- `components` : реєстр повторно використовуваних схем моделей (DTO), відповідей та схем безпеки.

Приклад опису контролера анотаціями Tag та Operation

```
@RestController
@RequestMapping("/api/cameras")
@Tag(name = "Камери спостереження", description = "Управління пристроями відеофіксації")
public class CameraController {

    @GetMapping("/{id}")
    @Operation(summary = "Отримати камеру за ID", description = "Повертає повну інформацію про камеру разом зі статусом потоку")
    public ResponseEntity<CameraResponse> getCameraById(@PathVariable Long id) {
        // логіка контролера
    }
}
```

Пояснення метаданих Tag та Operation

- Анотація `@Tag` об'єднує ендпоінти в логічні тематичні блоки в навігаційному меню Swagger UI.
- Анотація `@Operation` формує стислий заголовок та вичерпний опис бізнес-задачі окремого маршруту.
- Дозволяє розробникам клієнтських додатків швидко знаходити потрібні ендпоінти за їх призначенням.
- Покращує читабельність та структуру інтерактивної документації API.

Приклад опису можливих статус-кодів через ApiResponse

```
@PostMapping
@Operation(summary = "Створити нову камеру")
@ApiResponse({
    @ApiResponse(responseCode = "201", description = "Камеру успішно створено"),
    @ApiResponse(responseCode = "400", description = "Некоректні вхідні дані запиту"),
    @ApiResponse(responseCode = "409", description = "Камера з такою IP-адресою вже існує")
})
public ResponseEntity<CameraResponse> createCamera(@Valid @RequestBody CreateCameraRequest request) {
    // логіка збереження
}
```

Пояснення значущості каталогу статус-кодів

- Анотація `@ApiResponse` фіксує перелік усіх можливих результатів виконання бізнес-операції.
- Документування статусів помилок (400, 404, 409) усуває непередбачуваність для розробників інтеграцій.
- Формує повний контракт, на який орієнтуються фахівці під час написання автоматизованих тестів.
- Дозволяє генерувати якісні клієнтські SDK з повною типізацією помилок.

Приклад опису DTO з анотаціями валідації та Schema

```
@Schema(description = "Дані для реєстрації нової камери")
public record CreateCameraRequest(
    @Schema(description = "Назва камери", example = "Вхідна зона 1")
    @NotBlank @Size(min = 3, max = 50)
    String name,

    @Schema(description = "Мережева адреса", example = "192.168.1.120")
    @Pattern(regexp = "^\\d{1,3}(\\.\\d{1,3}){3}$")
    String ipAddress,

    @Schema(description = "Порт для RTSP потоку", example = "554")
    @Min(1) @Max(65535)
    int port
) {}
```

Пояснення генерації схеми моделі даних

- Анотація `@Schema` задає людиночитний опис сутності та практичні приклади значень через атрибут `example`.
- Обмеження Jakarta Validation автоматично транслюються у властивості JSON Schema (`minLength`, `pattern`, `minimum`).
- Клієнти бачать правила перевірки даних безпосередньо у документації до відправки першого HTTP-запиту.
- Синхронізує серверні валідаційні обмеження з клієнтськими формами.

Приклад обробника винятків із форматом ProblemDetail

```
@ExceptionHandler(CameraNotFoundException.class)
@ApiResponse(responseCode = "404", description = "Камеру не знайдено",
    content = @Content(schema = @Schema(implementation = ProblemDetail.class)))
public ProblemDetail handleNotFound(CameraNotFoundException ex) {
    ProblemDetail problem = ProblemDetail.forStatusAndDetail(HttpStatus.NOT_FOUND, ex.getMessage());
    problem.setTitle("Ресурс відсутній");
    problem.setType(URI.create("https://api.example.com/errors/not-found"));
    return problem;
}
```

Приклад конфігурації схеми безпеки BearerAuth

```
@Configuration
public class OpenApiSecurityConfiguration {

    @Bean
    public OpenAPI customOpenAPI() {
        return new OpenAPI()
            .info(new Info().title("CCTV Service API").version("1.0.0"))
            .addSecurityItem(new SecurityRequirement().addList("BearerAuth"))
            .components(new Components().addSecuritySchemes("BearerAuth",
                new SecurityScheme()
                    .name("BearerAuth")
                    .type(SecurityScheme.Type.HTTP)
                    .scheme("bearer")
                    .bearerFormat("JWT"))));
    }
}
```

Пояснення роботи механізму авторизації у Swagger UI

- Реєстрація компонента `SecurityScheme` додає інтерактивну кнопку `Authorize` у верхню панель Swagger UI.
- Розробник або тестувальник вводить JWT-токен один раз, після чого він автоматично підставляється у всі запити.
- Клієнтські запити автоматично містять заголовок `Authorization: Bearer <token>` під час тестування з браузера.
- Забезпечує повноцінне інтерактивне тестування захищених методів API.

Антипатерни в проєктуванні документації API

Типові помилки, які нівелюють цінність схем:

- Використання нетипізованих структур `Map<String, Object>`: позбавляє клієнта можливості дізнатися поля та типи моделі.
- Повернення фіктивного статусу 200 OK при збоях: приховування помилок у тілі відповіді порушує стандарт REST.
- Відсутність прикладів (`example`): розробникам інтеграцій доводиться вгадувати формати дат, телефонів та ідентифікаторів.
- Витік внутрішніх сутностей бази даних (`@Entity`): загрожує безпеці та створює циклічні залежності в документації.

Тестування за межами браузера на платформі Postman

Swagger UI підходить для швидкого ознайомлення, але має обмеження:

- Не зберігає історію тестових сценаріїв та взаємозв'язок між послідовними запитами.
- Не дозволяє запускати перевірки автоматизовано у пайплайнах неперервної інтеграції.
- Postman дозволяє імпортувати схему OpenAPI через URL `/v3/api-docs` та отримати готову колекцію всіх запитів проєкту за лічені секунди.

Приклад збереження ідентифікатора у змінну Postman

```
// Збереження отриманого ID створеного ресурсу в змінну оточення  
pm.environment.set("currentCameraId", pm.response.json().id);
```

Пояснення структури та змінних середовищ у Postman

- **Колекції (Collections):** логічні каталоги з HTTP-запитами, згрупованими за функціональними модулями системи.
- **Оточення (Environments):** набори параметрів (`baseUrl` , `jwtToken` , `camera_id`) для легкого перемикання між серверами.
- Динамічне збереження ідентифікатора дозволяє будувати наскрізні ланцюжки залежних інтеграційних сценаріїв.
- Скрипти виконуються автоматично в пісочниці Postman після успішного виконання запиту.

Приклад автоматизованих перевірок у Postman Tests

```
pm.test("Статус відповіді 201 Created", function () {  
    pm.response.to.have.status(201);  
});  
  
pm.test("Відповідь містить згенерований ID", function () {  
    const jsonData = pm.response.json();  
    pm.expect(jsonData.id).to.be.a('number');  
    pm.expect(jsonData.name).to.eql("Вхідна зона 1");  
});
```

Пояснення написання автоматизованих тестів у Postman

- Вкладка **Tests** містить JavaScript-інструкції, які виконуються рушієм негайно після отримання відповіді.
- Бібліотека перевірок спирається на синтаксис assertions BDD та перевіряє схему, статус-коди й швидкість відповіді.
- Усуває ручну візуальну перевірку тіла JSON розробником та формує основу для автоматизованої валідації.
- Надає можливість перевіряти заголовки відповіді та валідність бізнес-полів моделі.

[Документація: Postman Test Scripts](#)

Виконання тестів у консолі через Newman CLI

Newman є офіційним консольним рушієм для виконання колекцій Postman:

```
npx newman run camera_collection.json -e environment.json
```

- Не потребує графічного інтерфейсу та працює в терміналі операційної системи.
- Зчитує збережену колекцію, послідовно виконує всі запити та проводить перевірку assertions.
- Генерує зведений текстовий та HTML-звіт про проходження або падіння тестів.

Приклад пайплайну тестування контрактів у GitHub Actions

```
name: API Regression Tests
on: [push, pull_request]

jobs:
  test-api:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Start Spring Boot Service
        run: ./mvnw spring-boot:start
      - name: Run Newman Tests
        run: npx newman run ./tests/collection.json -e ./tests/env.json
```

Пояснення автоматизації тестування у пайплайнах CI/CD

- Запуск сервісу Spring Boot та прогін тестової колекції автоматично ініціюються на кожному Push або Pull Request.
- Помилка хоча б одного assertion у сценарії Newman завершує раннер із кодом збою та блокує небезпечне злиття гілок.
- Забезпечує безперервний контроль сумісності API перед публікацією релізу в промислове середовище.
- Знижує ризик регресій та гарантує виконання домовленостей за контрактом API.

Приклад запуску тестів через консольний рушій Newman

```
# Прогін тестів із формуванням звіту JUnit XML та перериванням при збої  
npx newman run cctv_collection.json -e cctv_env.json --reporters cli,junit,htmlextra --reporter-junit-export ./results/newman-report.xml --bail
```

Пояснення прапорців та параметрів виконання Newman

- Репортери (`--reporters`): одночасне виведення в термінал (`cli`), генерація XML-звіту для JUnit (`junit`) та інтерактивного звіту (`htmlextra`).
- Прапорець переривання (`--bail`): негайно зупиняє подальше виконання сценаріїв при першому ж збої для оптимізації часу пайплайну.
- Коди завершення (Exit Codes): при виявленні невідповідності повертається `exit code 1` , що сигналізує провал кроку збірки.
- Інтегрується з будь-якими системами неперервної інтеграції (Jenkins, GitLab CI, GitHub Actions).

[Документація: Newman CLI Integration](#)

Підсумки та висновки лекції

- Стартери та автоконфігурація Spring Boot забезпечують модульність кодової бази та усувають шаблонний код.
- Журналювання через SLF4J та MDC надає спостережуваність і наскрізну простежуваність подій без витоку персональних даних.
- Специфікація OpenAPI у парі зі SpringDoc утворює живий машиночитний контракт сервісу.
- Автоматизація тестування через Postman та Newman замикає контур гарантії якості бекенду на етапі CI/CD.

Рекомендовані першоджерела та корисні посилання

Офіційна документація та міжнародні стандарти для поглибленого вивчення:

- [Spring Boot Developing Auto-configuration](#)
- [Spring Boot External Configuration Specification](#)
- [Spring Boot Logging & Structured Logging Guide](#)
- [OpenAPI Specification v3.1.0](#)
- [SpringDoc OpenAPI Official Documentation](#)
- [Postman & Newman CLI Automated Testing Guide](#)

Індивідуальне завдання

Налаштування логування та автоматизація перевірки через Newman:

1. Реалізувати `OncePerRequestFilter` , що додає `traceId` у MDC та заголовок відповіді `X-Trace-Id` з обов'язковим очищенням у блоці `finally` .
2. Налаштувати формат виводу логів у `application.yml` з `[%X{traceId}]` та використати параметризований вивід `{}` у SLF4J.
3. Створити в Postman запит до ендпоінта та додати перевірку статусу 200 і наявності заголовка `X-Trace-Id` .
4. Експортувати колекцію у `collection.json` та виконати автоматизований прогін через команду `npx newman run collection.json` .

Групове завдання

Вимоги до спільного автоконфігураційного модуля команди:

1. Створити спільний стартер або конфігураційний модуль у командному репозиторії, що надає спільний функціонал (наприклад, єдиний формат відповідей або сповіщення).
2. Реалізувати ієрархію налаштувань модуля з підтримкою профілів середовищ `application-dev` та `application-prod` (`.properties` або `.yml`).
3. Додати автоматичні тести автоконфігурації за допомогою `ApplicationContextRunner` для перевірки наявності бінів за різних прапорців.

Групове завдання

Вимоги до промислової підсистеми журналювання проєкту:

1. Створити конфігураційний файл `logback-spring.xml` із налаштуванням `RollingFileAppender` (ротація за днями та архівація старих файлів).
2. Реалізувати логіку маскування конфіденційних полів (паролі, платіжні реквізити) для запобігання витоку персональних даних у лог-файли.
3. Провести аудит викликів у сервісах команди та переконатися у відсутності приховування винятків у блоках `catch`.

Групове завдання

Вимоги до документації та автоматизації через Newman CLI:

1. Задokumentувати ендпоінти командного проєкту у Swagger UI з мінімум 1 прикладом схеми моделі (`example`).
2. Підготувати колекцію тестів Postman для 1 ключового інтеграційного сценарію та зберегти її в каталозі проєкту.
3. Налаштувати успішне автоматизоване виконання сценарію через команду `npx newman run collection.json -e env.json`.